

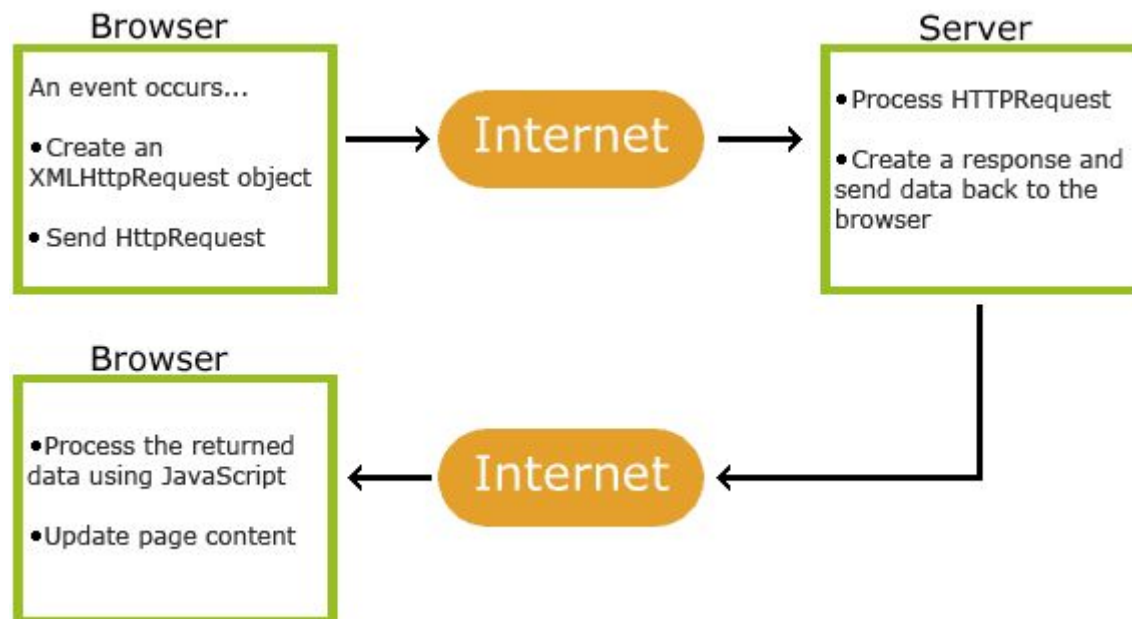
Ajax

Asynchronous JavaScript and XML

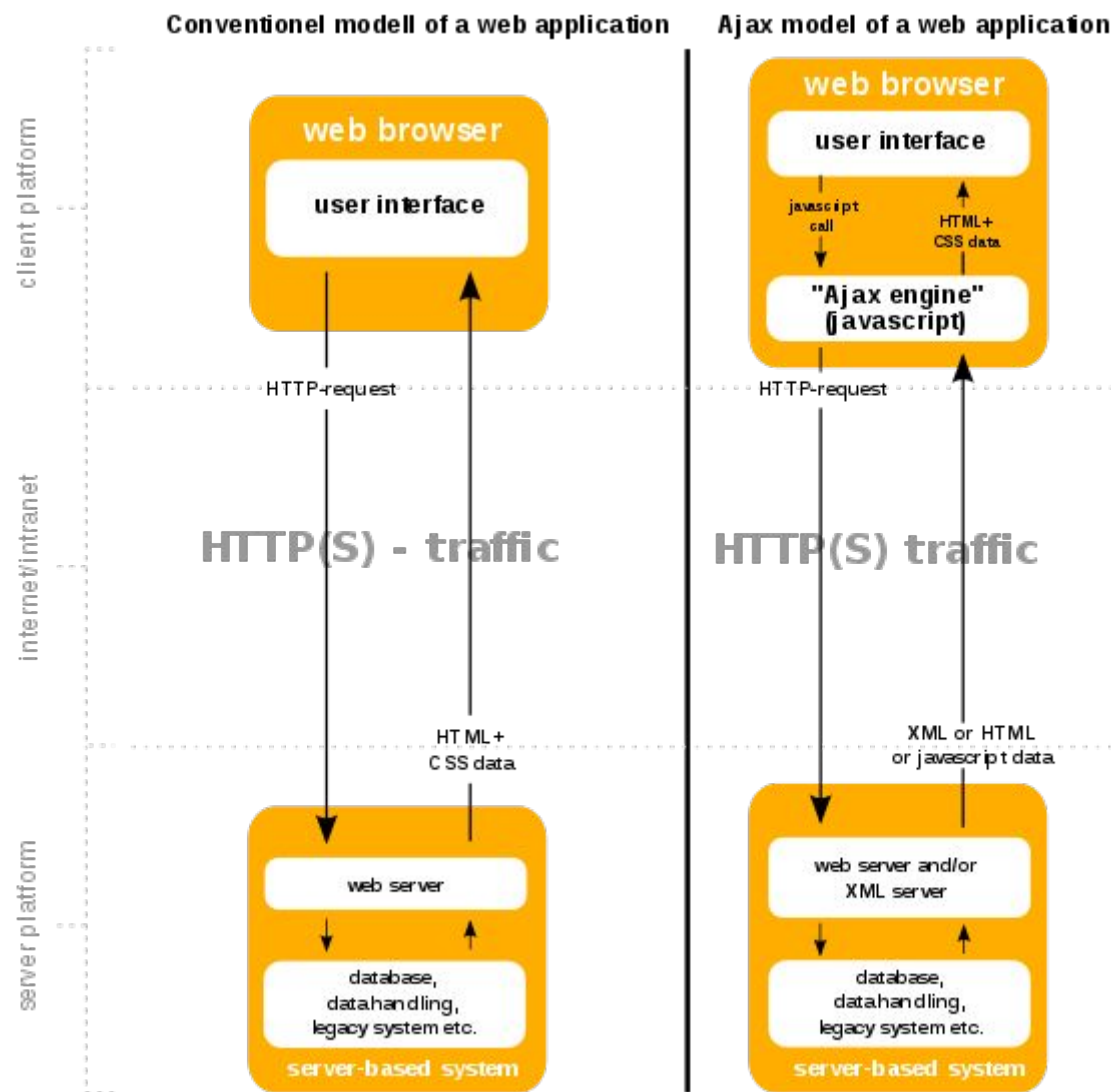
dr. Matej Šprogar

Zakaj Ajax?

Ajax je tehnika izdelave hitrih in dinamičnih spletnih strani, ki omogoča asinhrono posodobitev spletne strani brez ponovnega nalaganja celotne strani. Ideja je v izmenjavi **manjših** količin podatkov s strežnikom, ki se dogaja v **ozadju**.



Primerjava delovanja



Kaj potrebuje

Ajax potrebuje

- JavaScript (drži skupaj Ajax operacijo)
 - DOM (predstavlja strukturo XML in HTML dokumentov)
 - XMLHttpRequest (JavaScript objekt, ki izvede *asinhrono* komunikacijo s strežnikom)
 - JSON/XML za prenos podatkov
-
- + kompatibilen brskalnik
 - + strežniški software, ki odgovarja na zahteve

JSON - JavaScript Object Notation

Preprost in človeku razumljiv datotečni format za izmenjavo podatkov o objektih v obliki *polj* in parov *atribut:vrednost*.

Izhaja iz JavaScripta, vendar je uporabljan v vseh jezikih, uporablja UTF-8 znakovno kodiranje. Zelo podoben JavaScript objektom, razen da so stringi v dvojnih narekovajih, ni komentarjev...

JS uporaba:

```
// iz stringa → object  
obj = JSON.parse(str);
```

```
// iz objecta → string  
str = JSON.stringify(obj);
```

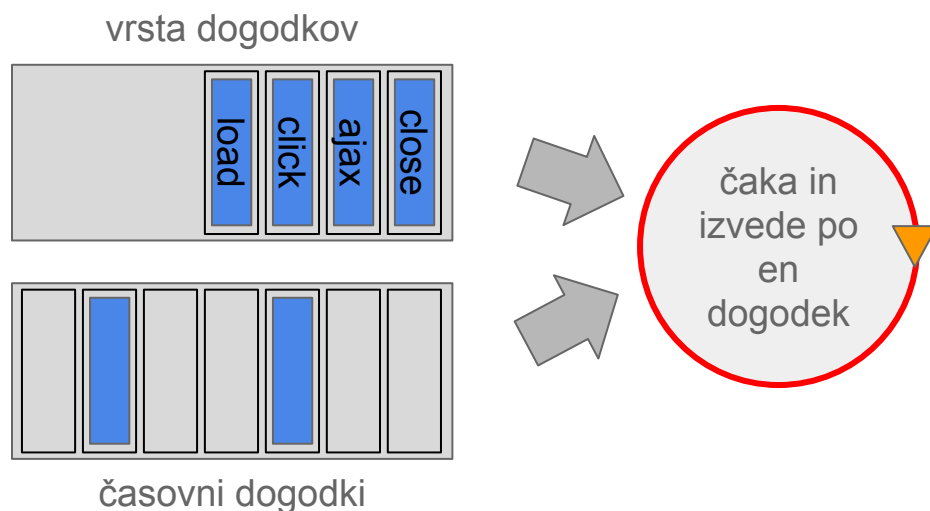
```
{  
  "avtor": "Douglas Adams",  
  "opus": [  
    { "naslov": "Štoparski vodnik po galaksiji",  
      "leto": 1979 },  
    { "naslov": "Restavracija na koncu vesolja",  
      "leto": 1980 }  
  ]  
}
```

Kaj Ajax omogoča

- pošiljanje posodobitev na stran (*Ajax push*)
 - status, vreme, *chat*, ...
- interakcijo z uporabnikom glede vnosov
 - poglobljanje (npr. država>mesto), *autocomplete*, ...
- hramba uporabniških podatkov na strežniku
 - Google docs, ...
- izvajanje strežniških ukazov v okviru iste strani
 - glasovanje, email portali, ...

JS se izvaja v eni niti (*single thread*)

- Samo ena funkcija se izvaja hkrati (*singlethreaded*)
 - vse ostalo čaka v vrsti
 - ni težav zaradi sočasnega dostopa do podatkov
- Asinhrona funkcija se ne izvede takoj (ob dogodku)
- **Zakasnitev v sinhroni funkciji zakasni vse ostale**
- Brskalnik/dokument ne izkorišča* večjedrnega procesorja direktno, asinhronost je dosežena s proženjem dogodkov



single thread event loop

izvede časovni dogodek, ki mu je potekel rok, ali dogodek na začetku vrste

vrsta se polni ob uporabniških akcijah, strežniškem odzivu ali dogodkih

Koraki Ajax operacije

1. generiranje XMLHttpRequest objekta v JavaScriptu
2. konfiguracija XMLHttpRequest objekta
3. pošiljanje zahteve na strežnik na *asinhroni* način
4. strežnik sprocesa zahtevo in vrne odgovor
5. browser kliče konfigurirano *callback* funkcijo
6. posodabljanje HTML DOM-a

Izumrla koda

```
<!DOCTYPE html>
<html>
<body>

<div id="demo">
  <h2>Let AJAX change this text</h2>
  <button type="button" onclick="loadDoc()">Change Content</button>
</div>

<script>
function loadDoc() {
  var xhr = new XMLHttpRequest();
  xhr.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
      // s send() poslana zahteva je sedaj zaključena; sledi prikaz rezultata
      document.getElementById("demo").innerHTML = this.responseText;
    }
  };
  xhr.open("GET", "ajax_info.txt", true);    // send() bo asinhroni klic
  xhr.send();
  // šele ko bo zahteva obdelana na strežniku, bo spremenjen readyState zahteve na 4
}
</script>

</body>
</html>
```

readyState:

0: UNSENT	objekt narejen, <i>open()</i> še ni bil izveden
1: OPENED	vzpostavljena povezava z <i>open()</i>
2: HEADERS_RECEIVED	<i>send()</i> je bil klican, headerji+status na razpolago
3: LOADING	Obdelava in nalaganje; <i>responseText</i> je delno izpolnjen
4: DONE	operacija zaključena, rezultat je pripravljen

VČASIH:

https://www.w3schools.com/js/tryit.asp?filename=tryjs_ajax_first

```
<!DOCTYPE html>
<html>
<body>
<div id="demo">
  <h2>Let AJAX change this text</h2>
  <button type="button" onclick="loadDoc()">Change Content</button>
</div>
<script>
function loadDoc() {
  var xhr = new XMLHttpRequest();
  xhr.onload = function() {
    // odziv na s send() poslano zahtevo je bil uspešno naložen
    document.getElementById("demo").innerHTML = this.responseText;
  };
  xhr.open("GET", "ajax_info.txt", true); // parameter true pomeni, da bo send() asinhron
  xhr.send();
}
</script>
</body>
</html>
```

DANES: JavaScript Fetch API (→ *Promise*)

```
<!DOCTYPE html>
<html>
<body>
  <script>
    fetch('example.json')
      .then(function(response) {
        if (!response.ok) {
          throw new Error(response.status);
        }
        return response.json();
      })
      .then(myJson => {
        document.getElementById('msg').innerHTML = myJson.sporocilo;
      })
      .catch(function(error) {
        console.log('Nekaj je šlo narobe: \n', error);
      });
  </script>
  <p id="msg">Placeholder</p>
</body>
</html>
```

```
// example.json datoteka:
{"sporocilo": "Odgovor je 42, ampak kaj je vprašanje?"}
```

Primer: html + js + fetch API

HTML

```
<script>
function nalozi_nalogo(id_naloge)
{
    let opc = moje_opcije();

    post_it('/abc/naloga', {
        'idn' : id_naloge,
        'opcije' : opc
    })
    .then(podatki => {
        let vsebina =
            document.getElementById('vsebina');
        vsebina.innerHTML = podatki.vsebina;
    });
}
</script>
```

JS

```
function post_it(url, data = {}) {
    return fetch(url, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json;
charset=utf-8' },
        body: JSON.stringify(data) })
    .then(function(response) {
        if (!response.ok)
            throw Error(response.statusText);
        return response.json();
    });
}

function get_it(url, data = {}) {
    const params = [];
    for(let key in data)
        params.push(encodeURIComponent(key) + '=' +
            encodeURIComponent(data[key]));

    return fetch(url + '?' + params.join('&'))
    .then(function(response) {
        if (!response.ok)
            throw Error(response.statusText);
        return response.json();
    });
}
```

JQuery Ajax API

`$.getScript(url, callback)`

pridobi in izvede skripto na URL, nato kliče *callback*

`e.load(url)`

pridobi vsebino na URL in jo vstavi v element *e*

`$.getJSON(url, data, callback)`

pošlje podatke strežniku na URL in strežnikov

JSON odgovor posreduje *callback-u*

`$.get(url[, data][, callback][, type])`

`$.post(url[, data][, callback][, type])`

Naloži/pošlje podatke na strežnik

`$.ajax(...)`

najbolj univerzalna metoda na nizkem nivoju